

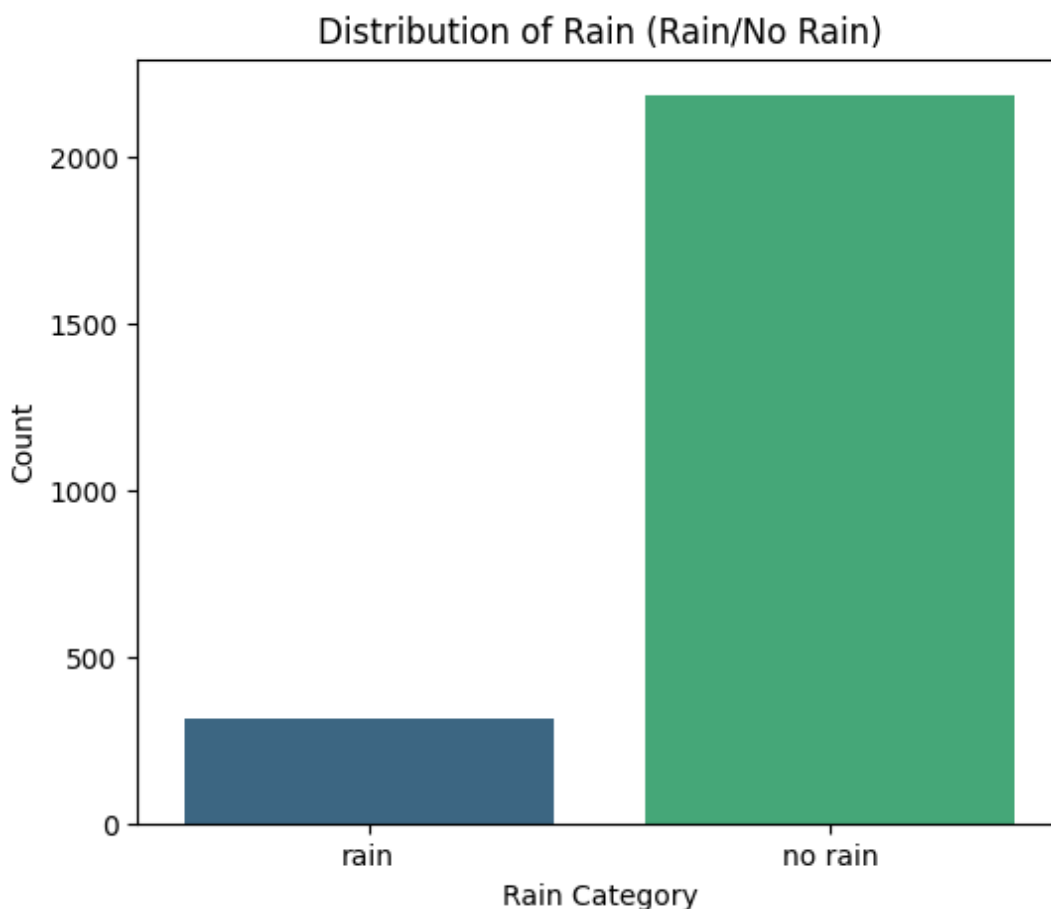
1 Phân tích khám phá dữ liệu (EDA)

1.1 Tổng quan bộ dữ liệu

Bộ dữ liệu được sử dụng trong nghiên cứu gồm 2500 quan sát và 6 biến, trong đó có 5 biến đầu vào định lượng liên tục là *Temperature*, *Humidity*, *Wind_Speed*, *Cloud_Cover*, *Pressure*, và 1 biến mục tiêu là *Rain*. Kết quả kiểm tra cấu trúc dữ liệu cho thấy toàn bộ các biến đầu vào đều ở dạng số thực (`float64`), trong khi biến mục tiêu *Rain* ở dạng chuỗi (`object`) với hai giá trị duy nhất là *rain* và *no rain*. Điều này cho thấy bài toán thuộc nhóm phân loại nhị phân (*binary classification*).

Ngoài ra, bộ dữ liệu không chứa giá trị thiếu và không có bản ghi trùng lặp. Cụ thể, số lượng giá trị thiếu của tất cả các biến đều bằng 0, và số lượng bản ghi trùng lặp cũng bằng 0. Điều này cho thấy dữ liệu có mức độ hoàn chỉnh cao, giúp giảm nhu cầu xử lý làm sạch dữ liệu phức tạp trước khi xây dựng mô hình.

1.2 Phân tích biến mục tiêu



Hình 1: Phân bố lớp của biến mục tiêu "rain"

Biến mục tiêu *Rain* bao gồm hai lớp là *rain* và *no rain*. Kết quả thống kê cho thấy lớp *no rain* chiếm 2186 quan sát, tương ứng 87.44%, trong khi lớp *rain* chỉ chiếm 314 quan sát, tương ứng 12.56%. Như vậy, bộ dữ liệu tồn tại hiện tượng mất cân bằng lớp (*class imbalance*) khá rõ rệt.

Đặc điểm này có ý nghĩa quan trọng đối với quá trình xây dựng và đánh giá mô hình. Nếu chỉ sử dụng chỉ số *accuracy*, mô hình có thể đạt kết quả cao ngay cả khi dự đoán thiên lệch về lớp đa số. Ví dụ, một mô hình luôn dự đoán *no rain* cho mọi quan sát vẫn có thể đạt độ chính xác xấp xỉ 87.44%. Do đó, ngoài *accuracy*, nghiên cứu cần sử dụng thêm các chỉ số như *precision*, *recall*, *F1-score* và *ROC-AUC* để phản ánh đầy đủ hơn chất lượng mô hình, đặc biệt đối với lớp thiểu số là *rain*.

1.3 Phân tích các biến thuộc tính

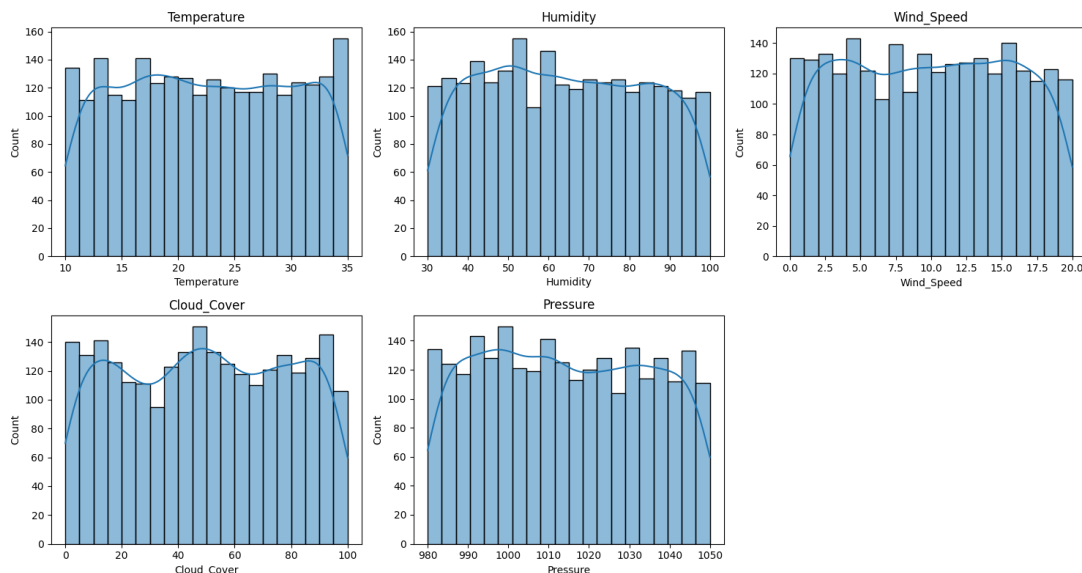
Statistics for each feature:							
	min	max	range	median	Q1	Q3	IQR
Temperature	10.001842	34.995214	24.993372	22.536448	16.359398	28.976476	12.617078
Humidity	30.005071	99.997481	69.992410	63.920797	47.339815	81.561021	34.221206
Wind_Speed	0.009819	19.999132	19.989313	9.908572	4.761909	14.948408	10.186499
Cloud_Cover	0.015038	99.997795	99.982757	49.488284	23.900016	75.324140	51.424124
Pressure	980.014486	1049.985593	69.971107	1013.433035	996.938630	1031.735067	34.796437

Hình 2: Thống kê mô tả của các biến thuộc tính

Năm biến đầu vào trong bộ dữ liệu đều là các biến định lượng liên tục và có phạm vi giá trị khác nhau. Kết quả thống kê mô tả cho thấy:

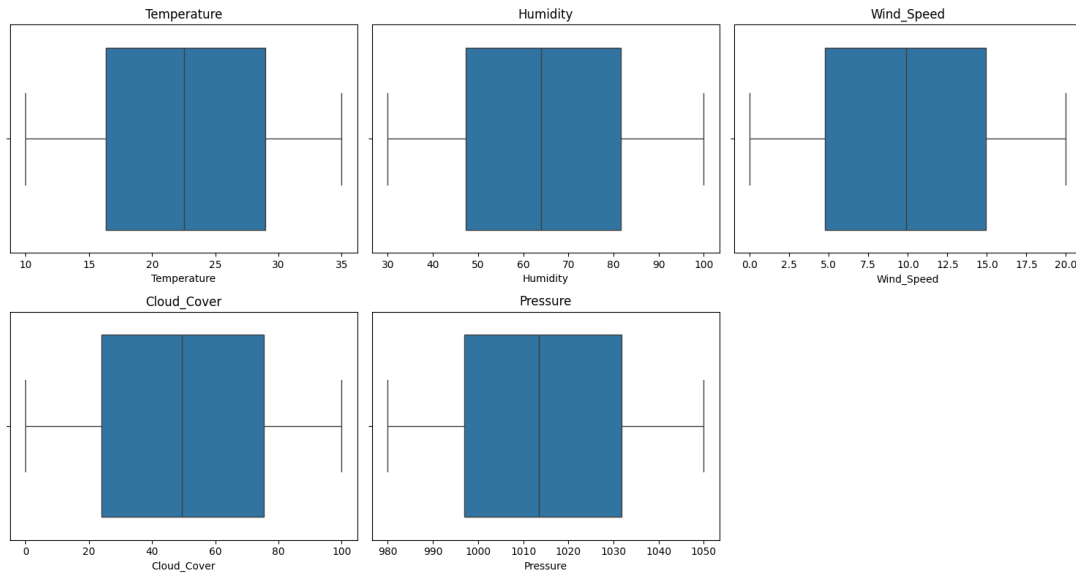
- **Temperature:** dao động từ 10.001842 đến 34.995214, với trung vị là 22.536448, *IQR* là 12.617078.
- **Humidity:** dao động từ 30.005071 đến 99.997481, với trung vị là 63.920797, *IQR* là 34.221206.
- **Wind_Speed:** dao động từ 0.009819 đến 19.999132, với trung vị là 9.908572, *IQR* là 10.186499.
- **Cloud_Cover:** dao động từ 0.015038 đến 99.997795, với trung vị là 49.488284, *IQR* là 51.424124.
- **Pressure:** dao động từ 980.014486 đến 1049.985593, với trung vị là 1013.433035, *IQR* là 34.796437.

Kết quả này cho thấy các biến đầu vào đều có mức độ biến thiên tương đối rõ, phản ánh sự đa dạng của các điều kiện thời tiết trong bộ dữ liệu. Trong đó, *Humidity*, *Cloud_Cover* và *Pressure* có biên độ dao động lớn, cho thấy khả năng chứa nhiều thông tin hữu ích cho bài toán phân loại. Đồng thời, các biến có thang đo khác nhau đáng kể; chẳng hạn *Pressure* có giá trị nằm trong khoảng xấp xỉ 980 đến 1050, trong khi *Wind_Speed* chỉ dao động trong khoảng từ 0 đến 20. Đặc điểm này đặc biệt quan trọng đối với các mô hình nhạy với thang đo của biến đầu vào như SVM hay Neural Network.



Hình 3: Biểu đồ Histogram của các biến thuộc tính

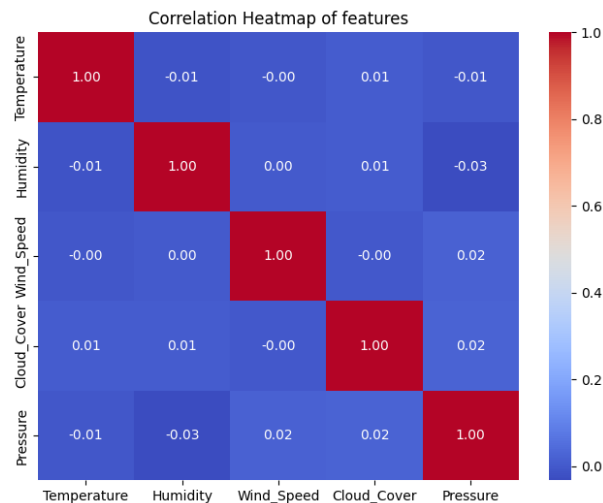
Kết quả trực quan hóa bằng histogram cho thấy các biến thuộc tính đều có phân bố trải khá rộng trên miền giá trị của mình và không xuất hiện hiện tượng lệch phân bố quá mạnh. Phần lớn các biến có dạng phân bố tương đối đồng đều, không tập trung quá mức vào một khoảng hẹp, cho thấy dữ liệu có độ phủ tốt và phản ánh được nhiều trạng thái khác nhau của điều kiện thời tiết.



Hình 4: Biểu đồ boxplot của các biến thuộc tính

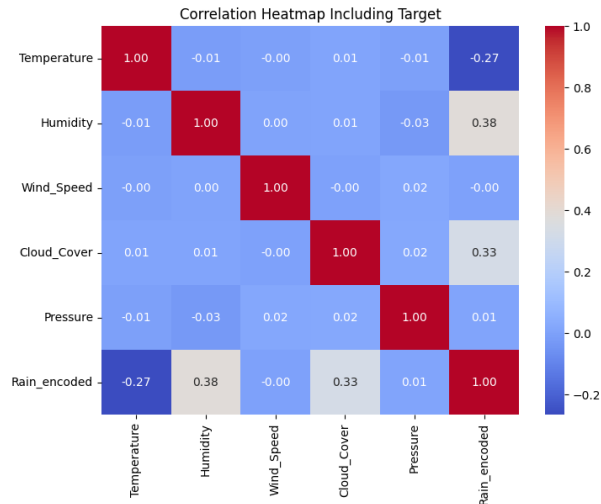
Bên cạnh đó, boxplot tổng quát của các biến không cho thấy sự xuất hiện rõ ràng của các ngoại lệ cực đoan theo tiêu chí IQR, do không có các điểm dữ liệu tách biệt rõ ngoài whisker. Điều này cho thấy bộ dữ liệu tương đối sạch ở mức đơn biến, và chưa có cơ sở để áp dụng các kỹ thuật xử lý outlier như cắt ngưỡng, loại bỏ quan sát hoặc biến đổi phân phối. Vì vậy, ở bước tiền xử lý, nghiên cứu không thực hiện can thiệp mạnh vào cấu trúc phân bố của các biến đầu vào.

1.4 Phân tích tương quan giữa các biến



Hình 5: Heatmap của các biến thuộc tính

Mã trận tương quan giữa các biến thuộc tính cho thấy hầu hết các hệ số tương quan đều rất nhỏ, dao động quanh giá trị 0. Điều này cho thấy các biến đầu vào gần như không có quan hệ tuyến tính mạnh với nhau. Từ góc độ mô hình hóa, kết quả này hàm ý rằng bộ dữ liệu không xuất hiện dấu hiệu đa cộng tuyến đáng kể giữa các đặc trưng, và chưa có cơ sở để loại bỏ biến do trùng lặp thông tin tuyến tính.



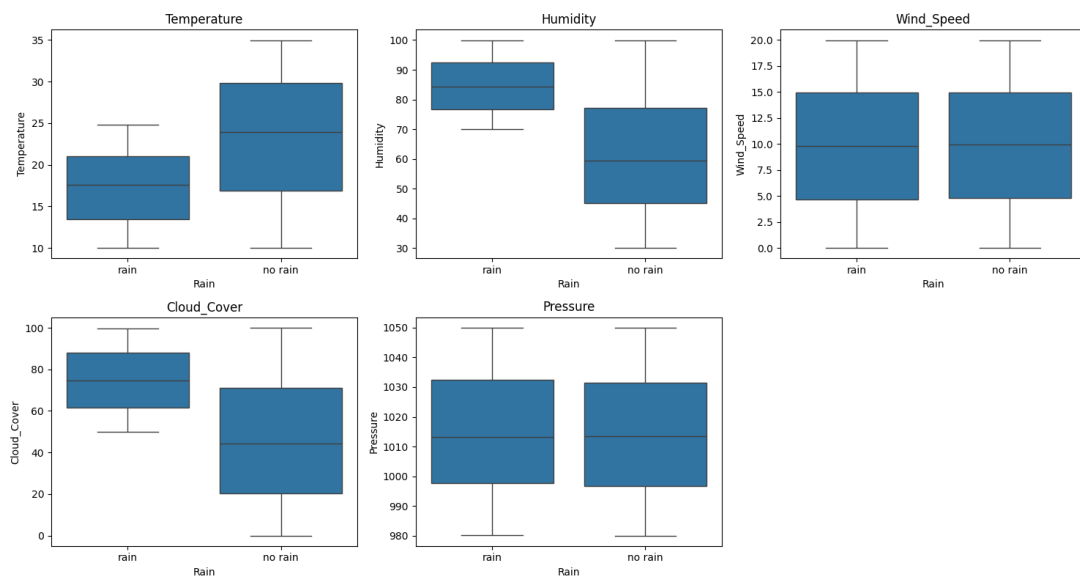
Hình 6: Heatmap của toàn bộ dữ liệu

Khi đưa biến mục tiêu *Rain* vào phân tích tương quan sau khi mã hóa nhị phân, kết quả cho thấy:

- *Humidity* có tương quan dương mức vừa với biến mục tiêu (0.38),
- *Cloud_Cover* có tương quan dương mức vừa (0.33),
- *Temperature* có tương quan âm mức vừa (-0.27),
- *Wind_Speed* và *Pressure* gần như không có tương quan tuyến tính đáng kể với biến mục tiêu.

Kết quả này cho thấy *Humidity*, *Cloud_Cover* và *Temperature* là ba biến có tín hiệu rõ hơn đối với bài toán dự đoán mưa, trong khi *Wind_Speed* và *Pressure* chưa thể hiện vai trò rõ rệt nếu chỉ xét theo mối liên hệ tuyến tính đơn biến.

1.5 Phân tích trực quan theo biến mục tiêu



Hình 7: Biểu đồ boxplot của các biến thuộc tính tương quan với biến mục tiêu

Biểu đồ hộp (*boxplot*) của từng biến theo hai lớp *rain* và *no rain* tiếp tục xác nhận kết quả từ ma trận tương quan. Cụ thể:

- *Humidity* có trung vị cao hơn rõ rệt ở nhóm *rain*,

- *Cloud_Cover* cũng có xu hướng cao hơn đáng kể ở nhóm *rain*,
- *Temperature* có xu hướng thấp hơn ở nhóm *rain*,
- *Wind_Speed* và *Pressure* không cho thấy sự khác biệt rõ rệt giữa hai lớp khi xét riêng lẻ.

Như vậy, từ cả phân tích định lượng và trực quan, có thể nhận định rằng *Humidity*, *Cloud_Cover* và *Temperature* là những biến có khả năng đóng góp nhiều hơn cho quá trình phân loại. Tuy nhiên, do các lớp vẫn còn chồng lấp ở một số vùng giá trị, việc phân loại chính xác vẫn cần dựa trên sự kết hợp của nhiều đặc trưng thay vì chỉ dựa vào một biến đơn lẻ.

1.6 Kết luận

Từ kết quả EDA, có thể rút ra một số nhận định quan trọng phục vụ cho việc lựa chọn và triển khai các mô hình trong nghiên cứu:

1. Bộ dữ liệu sạch, không có giá trị thiếu và không có bản ghi trùng lặp, do đó không cần áp dụng các bước làm sạch dữ liệu phức tạp trước khi huấn luyện mô hình.
2. Biến mục tiêu bị mất cân bằng lớp rõ rệt, vì vậy cần sử dụng chiến lược chia dữ liệu có phân tầng và các chỉ số đánh giá phù hợp như precision, recall, F1-score và ROC-AUC.
3. Các biến *Humidity*, *Cloud_Cover* và *Temperature* thể hiện khả năng phân biệt lớp rõ hơn, do đó được kỳ vọng sẽ đóng vai trò quan trọng hơn trong các mô hình phân loại.
4. Các biến đầu vào không có tương quan tuyến tính mạnh với nhau, cho thấy chưa có dấu hiệu đa cộng tuyến đáng kể và phù hợp để giữ lại toàn bộ biến trong giai đoạn xây dựng mô hình.
5. Các biến có thang đo khác nhau đáng kể, vì vậy cần chuẩn hóa dữ liệu đối với các mô hình nhạy với scale như SVM và Neural Network.
6. Với đặc điểm dữ liệu hiện tại, các mô hình cây như Decision Tree và Random Forest phù hợp để khai thác các ngưỡng phân tách và quan hệ phi tuyến; SVM phù hợp cho bài toán phân loại nhị phân có ít biến; Naive Bayes thích hợp như một mô hình đối chứng; còn Neural Network được sử dụng để kiểm tra thêm khả năng học quan hệ phi tuyến trong dữ liệu dạng bảng.

2 Tiền xử lý dữ liệu

2.1 Mục tiêu của bước tiền xử lý

Mục tiêu của bước tiền xử lý là chuyển đổi dữ liệu về dạng phù hợp cho quá trình huấn luyện và đánh giá mô hình, đồng thời giảm thiểu nguy cơ sai lệch, mất ổn định và rò rỉ dữ liệu (*data leakage*). Trong bài toán phân loại nhị phân với dữ liệu mất cân bằng lớp, tiền xử lý không chỉ giúp mô hình học hiệu quả hơn mà còn góp phần bảo đảm các chỉ số đánh giá phản ánh đúng năng lực dự đoán của mô hình.

Dựa trên kết quả EDA, các bước tiền xử lý trong nghiên cứu này tập trung vào:

- mã hóa biến mục tiêu,
- tách biến đầu vào và biến mục tiêu,
- chia dữ liệu thành tập huấn luyện và tập kiểm tra,
- thiết lập chiến lược đánh giá bằng *Stratified K-Fold Cross-validation*,
- chuẩn hóa dữ liệu đối với các mô hình nhạy với thang đo.

Các bước trên được lựa chọn nhằm tối ưu quá trình huấn luyện theo ba hướng chính: (i) bảo đảm dữ liệu đầu vào tương thích với mô hình học máy, (ii) tăng độ tin cậy của quá trình đánh giá, và (iii) cải thiện khả năng học của mô hình trên lớp thiểu số.

2.2 Mã hóa biến mục tiêu

Do biến mục tiêu *Rain* ban đầu ở dạng chuỗi với hai giá trị *rain* và *no rain*, nghiên cứu tiến hành mã hóa biến này về dạng nhị phân để phù hợp với các thuật toán học máy. Cụ thể:

$$no\ rain \rightarrow 0, \quad rain \rightarrow 1$$

Về mặt lý thuyết, hầu hết các mô hình phân loại trong *scikit-learn* yêu cầu nhãn đầu ra ở dạng rời rạc có thể xử lý bằng số. Việc mã hóa nhị phân giúp mô hình:

- biểu diễn rõ hai trạng thái của bài toán phân loại,
- thuận lợi cho việc tính toán xác suất dự đoán,
- hỗ trợ trực tiếp cho các thước đo đánh giá như *precision*, *recall*, *F1-score* và *ROC-AUC*.

Lý do lựa chọn cách mã hóa này là vì bài toán hiện tại chỉ có hai lớp và không cần sử dụng các chiến lược mã hóa phức tạp hơn như *one-hot encoding*.

2.3 Xác định tập biến đầu vào và biến mục tiêu

Sau khi mã hóa biến mục tiêu, tập biến đầu vào X được xác định gồm năm biến:

$$X = \{Temperature, Humidity, Wind_Speed, Cloud_Cover, Pressure\}$$

và biến mục tiêu y là:

$$y = Rain$$

Về mặt ký hiệu, với mỗi quan sát thứ i , ta có thể biểu diễn:

$$\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, x_3^{(i)}, x_4^{(i)}, x_5^{(i)}]$$

trong đó:

$$x_1 = Temperature, \quad x_2 = Humidity, \quad x_3 = Wind_Speed, \quad x_4 = Cloud_Cover, \quad x_5 = Pressure$$

Việc giữ lại toàn bộ năm biến đầu vào ở giai đoạn này là hợp lý vì:

- chưa có bằng chứng cho thấy tồn tại biến dư thừa rõ rệt,
- ma trận tương quan không cho thấy đa cộng tuyến mạnh,
- các mô hình phi tuyến như Decision Tree và Random Forest vẫn có thể khai thác thông tin từ các biến có tương quan tuyến tính thấp.

2.4 Chia dữ liệu train/test

Dữ liệu được chia thành tập huấn luyện và tập kiểm tra theo tỷ lệ 80/20. Quá trình chia sử dụng phương pháp *stratified split* theo biến mục tiêu nhằm bảo đảm tỷ lệ giữa hai lớp *rain* và *no rain* được duy trì tương đối ổn định trong cả hai tập dữ liệu.

Giả sử toàn bộ tập dữ liệu là D , quá trình chia dữ liệu tạo ra:

$$D = D_{\text{train}} \cup D_{\text{test}}, \quad D_{\text{train}} \cap D_{\text{test}} = \emptyset$$

với:

$$|D_{\text{train}}| \approx 0.8|D|, \quad |D_{\text{test}}| \approx 0.2|D|$$

Việc sử dụng phân tầng là cần thiết do bộ dữ liệu có hiện tượng mất cân bằng lớp rõ rệt. Nếu chia ngẫu nhiên thông thường, một số tập con có thể bị lệch mạnh về tỷ lệ lớp, dẫn đến đánh giá mô hình không ổn định hoặc không phản ánh đúng đặc trưng của dữ liệu gốc.

Lý do lựa chọn chiến lược này là:

- giữ lại một tập kiểm tra độc lập để đánh giá cuối cùng khả năng khái quát hóa của mô hình,
- tránh sử dụng toàn bộ dữ liệu cho quá trình lựa chọn mô hình và tuning, vốn có thể làm kết quả đánh giá trở nên lạc quan giả tạo,
- bảo đảm lớp thiểu số vẫn được đại diện hợp lý trong cả tập huấn luyện và tập kiểm tra.

2.5 Chiến lược đánh giá bằng Stratified K-Fold Cross-validation

Trên tập huấn luyện, nghiên cứu sử dụng *Stratified K-Fold Cross-validation* với $K = 10$ để đánh giá mô hình. Phương pháp này chia tập huấn luyện thành 10 fold, trong đó mỗi lần lặp sử dụng 9 fold để huấn luyện và 1 fold để xác thực. Việc phân tầng giúp duy trì tỷ lệ giữa hai lớp trong từng fold, từ đó giảm sự biến động giữa các lần đánh giá.

Việc lựa chọn *Stratified K-Fold Cross-validation* thay vì các phương pháp khác là vì:

- tận dụng dữ liệu hiệu quả hơn so với một lần chia train/validation cố định,
- giảm phụ thuộc vào tính ngẫu nhiên của một lần phân chia dữ liệu,
- đặc biệt phù hợp với dữ liệu mất cân bằng lớp, vì mỗi fold đều duy trì được cấu trúc phân bố lớp tương tự dữ liệu gốc,
- cung cấp ước lượng ổn định hơn về hiệu năng mô hình trong quá trình so sánh và tuning.

2.6 Chuẩn hóa dữ liệu

Do các biến đầu vào có thang đo khác nhau đáng kể, bước chuẩn hóa dữ liệu được xem xét đối với các mô hình nhạy với scale, chẳng hạn như:

- Support Vector Machine (SVM),
- Neural Network.

Trong nghiên cứu này, chuẩn hóa Z-score được áp dụng để đưa các biến đầu vào về cùng thang đo. Với mỗi biến x , giá trị chuẩn hóa được xác định bởi:

$$z = \frac{x - \mu}{\sigma}$$

trong đó:

- μ là giá trị trung bình của biến trên tập huấn luyện,
- σ là độ lệch chuẩn của biến trên tập huấn luyện.

Sau chuẩn hóa, dữ liệu có trung bình xấp xỉ 0 và độ lệch chuẩn xấp xỉ 1. Về mặt lý thuyết, việc chuẩn hóa giúp tránh hiện tượng biến có độ lớn lớn hơn chi phối quá trình tối ưu, cải thiện tốc độ hội tụ của mô hình, làm cho không gian đặc trưng trở nên cân bằng hơn đối với các mô hình dựa trên khoảng cách hoặc tối ưu gradient.

Lý do lựa chọn chuẩn hóa Z-score thay vì các phương pháp khác là vì:

- dữ liệu không có outlier cực đoan rõ rệt ở mức tổng quát,
- Z-score là phương pháp phổ biến và phù hợp với các mô hình như SVM và Neural Network,
- dễ tích hợp trong pipeline của *scikit-learn*.

Ngược lại, đối với các mô hình dạng cây như *Decision Tree* và *Random Forest*, bước chuẩn hóa không bắt buộc, do các mô hình này không phụ thuộc mạnh vào độ lớn tuyệt đối của đặc trưng mà chủ yếu dựa trên các ngưỡng phân chia.

2.7 Xử lý mất cân bằng lớp

Do bộ dữ liệu có hiện tượng mất cân bằng lớp, nghiên cứu không chỉ dựa vào *accuracy* mà còn chú trọng các chỉ số như *precision*, *recall*, *F1-score* và *ROC-AUC*.

Các độ đo này được xác định như sau:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong đó:

- TP là số mẫu dương tính được dự đoán đúng,
- FP là số mẫu âm tính bị dự đoán nhầm thành dương tính,
- FN là số mẫu dương tính bị bỏ sót.

ROC-AUC là chỉ số đo diện tích dưới đường cong ROC, phản ánh khả năng phân biệt giữa hai lớp của mô hình phân loại nhị phân. Chỉ số này có giá trị trong khoảng từ 0 đến 1, trong đó giá trị càng cao cho thấy mô hình càng có khả năng xếp hạng đúng các quan sát thuộc lớp dương cao hơn lớp âm. Trong nghiên cứu này, *ROC-AUC* được sử dụng bên cạnh *accuracy*, *precision*, *recall*, *F1-score* nhằm đánh giá mô hình một cách toàn diện hơn, đặc biệt trong bối cảnh bộ dữ liệu có hiện tượng mất cân bằng lớp. Trong bối cảnh bài toán dự đoán mưa, việc nhận diện đúng lớp *rain* có ý nghĩa lớn hơn so với việc chỉ tối đa hóa tỷ lệ dự đoán đúng tổng thể.

Lý do không ưu tiên *accuracy* là vì với dữ liệu lệch lớp, mô hình có thể đạt độ chính xác cao ngay cả khi dự đoán kém trên lớp thiểu số.

Bên cạnh đó, các chiến lược như sử dụng *class weight* (làm tăng mức phạt cho lỗi trên lớp thiểu số) hoặc các kỹ thuật oversampling như SMOTE (tạo thêm các mẫu tổng hợp để tái cân bằng dữ liệu huấn luyện) có thể được cân nhắc ở giai đoạn thực nghiệm nhằm cải thiện khả năng nhận diện lớp thiểu số.

2.8 Kết luận phần tiền xử lý

Tóm lại, quy trình tiền xử lý trong nghiên cứu này tương đối gọn do dữ liệu ban đầu có chất lượng tốt. Tuy nhiên, mỗi bước đều được lựa chọn trên cơ sở lý thuyết và gắn với mục tiêu tối ưu quá trình huấn luyện. Cụ thể:

- mã hóa biến mục tiêu giúp dữ liệu tương thích với các thuật toán phân loại,
- chia train/test và áp dụng *Stratified K-Fold Cross-validation* giúp tăng độ tin cậy trong đánh giá,
- chuẩn hóa dữ liệu giúp các mô hình nhảy với thang đo học ổn định hơn,
- việc lựa chọn các độ đo như *precision*, *recall*, *F1-score* và *ROC-AUC* giúp phản ánh đầy đủ hơn hiệu quả mô hình trong bối cảnh dữ liệu mất cân bằng lớp.

Nhờ đó, dữ liệu được chuẩn bị ở dạng phù hợp cho các bước huấn luyện, đánh giá và so sánh mô hình trong các phần tiếp theo.

3 Huấn luyện và so sánh các mô hình

4 Random Forest

4.1 Giới thiệu mô hình

Random Forest (Rừng Ngẫu Nhiên) là một thuật toán học có giám sát thuộc nhóm học tổ hợp (ensemble learning), hoạt động bằng cách xây dựng nhiều cây quyết định trên các mẫu dữ liệu khác nhau và tổng hợp kết quả của chúng để đưa ra dự đoán cuối cùng. Trong bài toán phân loại, mỗi cây sẽ dự đoán một nhãn lớp, sau đó mô hình sử dụng cơ chế bỏ phiếu đa số (majority voting) để quyết định kết quả đầu ra. Về mặt lý thuyết, Random Forest giúp giảm hiện tượng quá khớp so với một cây quyết định đơn lẻ nhờ kết hợp nhiều cây có tính đa dạng cao.

Mô hình này được lựa chọn vì có nhiều ưu điểm phù hợp với bộ dữ liệu nghiên cứu. Thứ nhất, Random Forest hoạt động hiệu quả trên dữ liệu dạng bảng với các biến đầu vào là số liên tục. Thứ hai, mô hình không yêu cầu chuẩn hóa dữ liệu và có khả năng khai thác tốt các quan hệ phi tuyến cũng như tương tác giữa các thuộc tính. Thứ ba, Random Forest có độ ổn định cao và thường cho khả năng tổng quát hóa tốt hơn so với Decision Tree đơn lẻ. Ngoài ra, mô hình còn hỗ trợ đánh giá mức độ quan trọng của từng thuộc tính, qua đó giúp giải thích vai trò của các biến đầu vào trong quá trình dự đoán.

Đối với bài toán dự đoán mưa, Random Forest đặc biệt phù hợp vì kết quả EDA cho thấy một số biến như *Humidity*, *Cloud_Cover* và *Temperature* có tín hiệu phân loại rõ, trong khi các biến khác vẫn có thể đóng vai trò hỗ trợ khi kết hợp với nhau. Nhờ đó, Random Forest được xem là một mô hình mạnh và phù hợp để áp dụng trong nghiên cứu này.

4.2 Triển khai mô hình và tối ưu siêu tham số

Quá trình triển khai mô hình Random Forest trong nghiên cứu được thực hiện theo ba bước chính: xây dựng mô hình cơ sở (*baseline model*), xác định các siêu tham số cần điều chỉnh, và sử dụng *GridSearchCV* để lựa chọn cấu hình tối ưu.

4.2.1 Baseline model

Trước hết, nghiên cứu xây dựng một mô hình Random Forest cơ sở nhằm thiết lập mức hiệu năng ban đầu để làm mốc so sánh. Mô hình baseline được khởi tạo với các tham số cơ bản:

- `n_estimators = 100`
- `random_state = 42`

Trong đó:

- `n_estimators` là số lượng cây quyết định được xây dựng trong rừng. Giá trị 100 được sử dụng như một lựa chọn phổ biến, đủ để mô hình đạt mức ổn định tốt mà chưa làm chi phí tính toán tăng quá cao.
- `random_state = 42` được sử dụng để cố định yếu tố ngẫu nhiên, bảo đảm khả năng tái lập kết quả giữa các lần chạy.

Sau đó, do bộ dữ liệu có hiện tượng mất cân bằng lớp, nghiên cứu tiếp tục triển khai một biến thể của mô hình baseline với tham số:

- `class_weight = 'balanced'`

Tham số này cho phép mô hình tự động điều chỉnh trọng số của từng lớp theo tỷ lệ nghịch với tần suất xuất hiện của chúng trong dữ liệu. Nhờ đó, lớp thiểu số (*Có mưa*) được chú trọng hơn trong quá trình huấn luyện, giúp cải thiện khả năng nhận diện đúng lớp này.

Như vậy, ở giai đoạn đầu, nghiên cứu so sánh hai cấu hình cơ sở:

- Random Forest baseline thông thường;
- Random Forest baseline có `class_weight='balanced'`.

Việc xây dựng baseline model là rất cần thiết cho việc đánh giá và tối ưu mô hình, Vì nó cung cấp một hiệu năng ban đầu, giúp đánh giá và so sánh việc cải thiện các siêu tham số và hỗ trợ phân tích tác động của việc xử lý mất cân bằng lớp.

Bảng 1: So sánh kết quả Cross-validation giữa Random Forest Baseline và Random Forest Balanced

Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV ROC-AUC
RF Baseline	0.9990	1.0000	0.9920	0.9959	1.0000
RF Balanced	0.9990	1.0000	0.9920	0.9959	1.0000

Từ Bảng 1 có thể nhận thấy hai cấu hình *RF Baseline* và *RF Balanced* cho kết quả Cross-validation hoàn toàn tương đương trên tất cả các chỉ số đánh giá, bao gồm Accuracy, Precision, Recall, F1-score và ROC-AUC. Đây là một kết quả đáng chú ý vì về mặt lý thuyết, tham số `class_weight='balanced'` được đưa vào nhằm điều chỉnh trọng số của các lớp theo tỷ lệ nghịch với tần suất xuất hiện của chúng trong dữ liệu, từ đó buộc mô hình chú ý nhiều hơn đến lớp thiểu số trong quá trình xây dựng cây. Điều này cho thấy các biến đầu vào đã mang tín hiệu phân tách lớp rõ ràng, mô hình có thể tự tạo ra các phép chia hiệu quả trong từng cây mà vẫn duy trì hiệu năng rất cao. Điều này phù hợp với các kết quả EDA trước đó, khi các biến như *Humidity*, *Cloud_Cover* và *Temperature* đã cho thấy mức độ liên hệ tương đối mạnh với biến mục tiêu. Điều đó cho thấy, Random Forest là một mô hình rất phù hợp với bộ dữ liệu hiện tại. Vì để đảm bảo tính hiệu quả đối với bộ dữ liệu mất cân bằng, *RF Balanced* sẽ được chọn để làm baseline model để so sánh và đánh giá kết quả cuối cùng

4.2.2 Hyperparameter Tuning

Sau bước baseline, nghiên cứu tiến hành tối ưu mô hình bằng cách điều chỉnh một số siêu tham số có ảnh hưởng lớn đến độ phức tạp và khả năng tổng quát hóa của Random Forest. Để lựa chọn cấu hình phù hợp, nghiên cứu sử dụng *GridSearchCV*. Đây là phương pháp tìm kiếm theo lưới, trong đó mô hình được huấn luyện và đánh giá trên tất cả các tổ hợp siêu tham số được xác định trước. Các siêu tham số được lựa chọn để kiểm tra bao gồm:

- `n_estimators`: Số lượng cây trong rừng. Giá trị lớn hơn thường giúp mô hình ổn định hơn, nhưng đồng thời làm tăng thời gian huấn luyện.
- `max_depth`: độ sâu tối đa của mỗi cây. Tham số này kiểm soát mức độ phức tạp của mô hình. Nếu cây quá sâu, mô hình có thể học quá sát dữ liệu huấn luyện và dẫn đến overfitting.
- `min_samples_split`: số mẫu tối thiểu cần có để một nút được phép tiếp tục phân chia. Giá trị lớn hơn giúp hạn chế việc chia quá chi tiết và làm mô hình bớt nhạy với nhiễu.
- `min_samples_leaf`: số mẫu tối thiểu cần có tại một nút lá. Tham số này giúp tránh việc tạo ra các lá quá nhỏ, vốn có thể làm mô hình học quá mức các mẫu cá biệt.

Các siêu tham số trên được lựa chọn vì chúng tác động trực tiếp đến độ sâu và cấu trúc của từng cây, mức độ phức tạp chung của toàn bộ mô hình, khả năng cân bằng giữa độ chính xác trên tập huấn luyện và khả năng tổng quát hóa trên dữ liệu mới.

Không gian tham số được kiểm tra bao gồm:

- `n_estimators`: {100, 200}
- `max_depth`: {None, 5, 10}
- `min_samples_split`: {2, 5, 10}
- `min_samples_leaf`: {1, 2, 4}

Quá trình tìm kiếm sử dụng `F1-score` để làm tiêu chí đánh giá. Việc sử dụng `F1-score` là phù hợp với bài toán mất cân bằng lớp, vì chỉ số này phản ánh đồng thời cả *precision* và *recall*, thay vì chỉ dựa trên *accuracy*. Cách tiếp cận này giúp tránh lựa chọn tham số theo cảm tính, bảo đảm quá trình tối ưu có hệ thống, nâng cao độ tin cậy của mô hình cuối cùng.

Kết quả mô hình sau Tuning qua 10 fold Cross-Validation

Bảng 2: So sánh hiệu năng giữa Baseline model và Final model qua 10 fold CV

Metric	Baseline model	Final model
Accuracy	0.9990	0.9990
Precision	1.0000	1.0000
Recall	0.9920	0.9920
F1-score	0.9959	0.9959
ROC-AUC	1.0000	1.0000

Bảng trên trình bày kết quả so sánh giữa mô hình Random Forest baseline và mô hình final sau khi tối ưu siêu tham số bằng GridSearchCV thông qua 10-fold cross-validation. Kết quả cho thấy hai mô hình đạt hiệu năng trung bình hoàn toàn tương đương trên tất cả các chỉ số đánh giá, bao gồm Accuracy, Precision, Recall, F1-score và ROC-AUC. Đồng thời, độ lệch chuẩn của các chỉ số cũng ở mức rất thấp, cho thấy cả hai mô hình đều có tính ổn định cao giữa các fold.

Bảng 3: Top 10 best hyperparameter configurations of the Random Forest model from GridSearchCV

No.	max_depth	min_samples_leaf	min_samples_split	n_estimators	Mean Test Score	Std Test Score
1	None	1	2	100	0.9959	0.0082
2	None	1	2	200	0.9959	0.0082
3	None	1	5	100	0.9959	0.0082
4	None	1	5	200	0.9959	0.0082
5	None	1	10	100	0.9959	0.0082
6	None	1	10	200	0.9959	0.0082
7	None	2	2	100	0.9959	0.0082
8	None	2	2	200	0.9959	0.0082
9	None	2	5	100	0.9959	0.0082
10	None	2	5	200	0.9959	0.0082

Điều này cho thấy quá trình tối ưu siêu tham số không tạo ra cải thiện rõ rệt về mặt định lượng trên bộ dữ liệu hiện tại. Nguyên nhân hợp lý là mô hình Random Forest baseline vốn đã đạt hiệu năng gần như tối ưu, trong bối cảnh các đặc trưng đầu vào có khả năng phân tách lớp rất mạnh. Do đó, vai trò chính của GridSearchCV trong nghiên cứu này không nằm ở việc nâng cao đáng kể metric, mà ở việc xác nhận một cách có hệ thống rằng cấu hình được lựa chọn là phù hợp và đáng tin cậy.

4.3 Đánh giá mô hình

Kết quả đánh giá mô hình:

- Accuracy: **1.00**
- ROC-AUC: **1.00**

Bảng 4: Classification Report

Metric	Lớp '0' (Không mưa)	Lớp '1' (Có mưa)
Precision	1.00	1.00
Recall	1.00	1.00
F1-score	1.00	1.00

Confusion Matrix:

$$\begin{bmatrix} 437 & 0 \\ 0 & 63 \end{bmatrix}$$

Kết quả đánh giá cho thấy mô hình Random Forest đạt hiệu năng rất cao trên tập kiểm thử, với Accuracy = **1.00**, ROC-AUC = **1.00**, đồng thời Precision, Recall và F1-score của cả hai lớp đều đạt **1.00**. Điều này cho thấy mô

hình có khả năng phân loại hoàn hảo trên bộ dữ liệu hiện tại, không chỉ đối với lớp đa số *Không mưa* mà còn đối với lớp thiểu số *Có mưa*, vốn là lớp có ý nghĩa nghiệp vụ quan trọng hơn trong bài toán dự báo.

Xét theo từng lớp, mô hình không tạo ra bất kỳ sai lệch nào trên tập kiểm thử:

- Lớp ‘1’ (Có mưa): Precision = 1.00, Recall = 1.00, F1-score = 1.00.
- Lớp ‘0’ (Không mưa): Precision = 1.00, Recall = 1.00, F1-score = 1.00.

Confusion Matrix của mô hình thể hiện:

- **True Negative (TN) = 437**: số ngày không mưa được dự đoán đúng;
- **True Positive (TP) = 63**: số ngày có mưa được dự đoán đúng;
- **False Positive (FP) = 0**: không có trường hợp báo động giả;
- **False Negative (FN) = 0**: không có trường hợp bỏ sót ngày mưa.

Từ góc độ bài toán mất cân bằng lớp, đây là một kết quả đặc biệt tích cực vì mô hình không chỉ đạt độ chính xác tổng thể cao mà còn duy trì được khả năng nhận diện hoàn hảo đối với lớp thiểu số. Giá trị ROC-AUC = 1.00 tiếp tục xác nhận rằng mô hình có khả năng phân biệt hoàn toàn giữa hai lớp *Có mưa* và *Không mưa*.

4.4 Mức độ quan trọng của đặc trưng

Một trong những ưu điểm quan trọng của mô hình Random Forest là khả năng cung cấp chỉ số đóng góp của các đặc trưng cho mô hình (*feature importance*), được tính dựa trên tổng mức giảm độ hỗn tạp (impurity) mà biến đó tạo ra trên toàn bộ các cây quyết định trong rừng. Điều đó cho phép đánh giá mức độ đóng góp tương đối của từng biến đầu vào trong quá trình dự báo. Đây được xem là một đặc trưng nổi bật của Random Forest, bởi bên cạnh khả năng phân loại mạnh, mô hình còn hỗ trợ diễn giải kết quả bằng cách chỉ ra biến nào thực sự có ảnh hưởng lớn đến các quyết định phân chia trong toàn bộ rừng cây.

Bảng 5: Kết quả phân tích feature importance của mô hình Random Forest

Feature	Importance	Tỷ lệ đóng góp (%)
Humidity	0.447342	44.73
Cloud_Cover	0.321863	32.19
Temperature	0.220801	22.08
Pressure	0.005272	0.53
Wind_Speed	0.004721	0.47

Về bản chất, trong quá trình huấn luyện, mỗi cây quyết định trong Random Forest sẽ lựa chọn các đặc trưng để phân chia dữ liệu tại từng nút sao cho tối đa hóa mức độ thuần nhất của các nút con. Khi một biến thường xuyên được chọn tại các nút phân chia quan trọng và tạo ra mức giảm độ hỗn tạp lớn, biến đó sẽ có giá trị *feature importance* cao. Do đó, chỉ số này phản ánh mức độ mà mô hình thực sự “dựa vào” một biến để đưa ra quyết định phân loại.

Kết quả phân tích cho thấy ba biến *Humidity*, *Cloud_Cover* và *Temperature* là các đặc trưng có mức độ quan trọng vượt trội, với giá trị *importance* lần lượt là 0.447342, 0.321863 và 0.220801. Tổng mức đóng góp của ba biến này đạt khoảng 98.90% tổng *importance*, cho thấy gần như toàn bộ năng lực phân loại của mô hình Random Forest được xây dựng dựa trên các đặc trưng này. Trong đó, *Humidity* là biến có ảnh hưởng lớn nhất, đóng vai trò trung tâm trong việc hình thành ranh giới phân loại giữa hai lớp *rain* và *no rain*.

Đáng chú ý, kết quả *feature importance* có sự nhất quán cao với các phát hiện từ giai đoạn phân tích khám phá dữ liệu (EDA). Cụ thể, trong EDA, các biến *Humidity*, *Cloud_Cover* và *Temperature* đã thể hiện sự khác biệt rõ rệt về phân phối giữa hai nhóm ngày mưa và không mưa, thông qua các biểu đồ trực quan và các thống kê mô tả. Điều này cho thấy ngay từ giai đoạn khám phá dữ liệu, các biến này đã mang thông tin phân biệt lớp mạnh. Khi được đưa vào mô hình Random Forest, các biến này tiếp tục được lựa chọn làm cơ sở cho các phép phân chia quan trọng, qua đó xác nhận rằng mô hình đang học dựa trên những đặc trưng có ý nghĩa thực tiễn thay vì các mẫu ngẫu nhiên.

Ngược lại, hai biến *Pressure* và *Wind_Speed* có mức độ quan trọng rất thấp, lần lượt là 0.005272 và 0.004721, chỉ chiếm khoảng 1.10% tổng *importance*. Điều này cho thấy hai biến này hầu như không đóng góp đáng kể vào các quyết định phân loại của mô hình. Kết quả này cũng phù hợp với EDA, khi hai biến trên không thể hiện sự tách biệt rõ ràng giữa hai lớp nếu xét riêng lẻ. Do đó, mặc dù vẫn được đưa vào mô hình, vai trò thực tế của chúng trong việc cải thiện hiệu năng dự báo là rất hạn chế.

Tổng thể, phân tích *feature importance* không chỉ giúp giải thích cơ chế hoạt động của mô hình Random Forest mà còn đóng vai trò như một bước kiểm chứng lại các giả thuyết được hình thành từ EDA. Sự nhất quán giữa hai giai đoạn này cho thấy mô hình có khả năng học đúng các đặc trưng quan trọng của dữ liệu, từ đó góp phần giải thích vì sao Random Forest đạt hiệu năng phân loại gần như hoàn hảo trên bộ dữ liệu nghiên cứu.